实验二 模型机组合部件的实现（一）（实验报告格式案例）

班级 计科 210X 姓名 甘晴void 学号 202108010XXX

一、实验目的

1．了解简易模型机的内部结构和工作原理。
2．熟悉译码器、运算器的工作原理。
3．分析模型机的功能，设计指令译码器。
4．分析模型机的功能，设计 ALU。

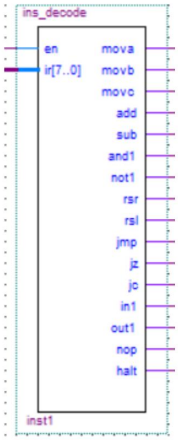
二、实验内容

1．指令译码器

指令译码器是根据指令系统表中的指令编码，对输入的 8 位指令进行解析，

汇编符号	功能	编码
MOV R1，R2	（R2）→ R1	1100 R1 R2
MOV M，R2	（R2）→（C）	1100 11 R2
MOV R1，M	（（C））→R1	1100 R1 11
ADD R1，R2	（R1）+（R2）→ R1	1001 R1 R2
SUB R1，R2	（R1）-（R2）→ R1	0110 R1 R2
AND R1，R2	（R1）&（R2）→ R1	1011 R1 R2
NOT R1	/（R1）→ R1	0101 R1 XX
RSR R1	（R1）循环右移一位→ R1	1010 R1 00
RSL R1	（R1）循环左移一位→ R1	1010 R1 11
JMP add	add → PC	0011 00 00，address
JZ add	结果为 0 时 add → PC	0011 00 01，address
JC add	结果有进位时 add → PC	0011 00 10，address
IN R1	（开关 7-0）→ R1	0010 R1 XX
OUT R2	（R2）→ 发光二极管 7-0	0100 XX R2
NOP		0111 00 00
HALT	停机	1000 00 00

判定是哪条指令，则对应指令的输出为 1，否则输出为 0。



指令译码器的输入输出引脚如上图所示。en 为使能信号，ir[7..0]是 8 位指令编码，输出是对应的 16 条指令。引脚之间的相互关系如下表所示：

表 2 指令译码器引脚关系

en	ir[7..0]	16 个输出信号
1	8 位的指令编码	指令编码对应的指令输出为 1，其它输出为 0
0	8 位的指令编码	不管 ir 为何值，16 个输出全为 0

2．ALU

算术逻辑运算类指令：

ADD R1，R2

SUB R1，R2

AND R1，R2

NOT R1

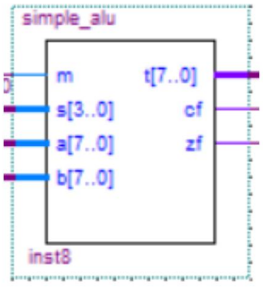
这类指令的执行过程为：

由 R2 的编码通过 RAA1、RAA0 从通用寄存器组 A 口读出 R2 的内容,由 R1 的编码通过 RWBA1、RWBA0 从通用寄存器组 B 口读出 R1 的内容，在 S3～S0 和 M 的控制下，实现运算，经移位逻辑送入总线 BUS；由/WE 控制和 R1 的编码选择 RWBA1、RWBA0，将 BUS 上的数据写入通用寄存器 R1。其中 ADD 和 SUB 指令影响状态位 Cf 和 Zf。

指令具体功能如下：

汇编符号	功能	编码
ADD R1，R2	（R1）+（R2）→ R1	1001 R1 R2
SUB R1，R2	（R1）-（R2）→ R1	0110 R1 R2
AND R1，R2	（R1）&（R2）→ R1	1011 R1 R2
NOT R1	/（R1）→ R1	0101 R1 XX

ALU 除了要完成 ADD、SUB、AND、NOT 运算外，还需在 MOVA、MOVB、RSR、RSL 和 OUT 五条指令执行时，提供将数据传送至总线的数据通路。ALU 模块的输入输出引脚如下图所示：



其中 m 和 s[3..0]是控制信号，控制 a[7..0]和 b[7..0]输入的数据进行什么操作，并将产生的结果输出到 t[7..0]、cf 和 zf。各引脚间的相互关系如下表所示：

表 3 ALU 引脚关系

m	s[3..0]	t[7..0]	cf	zf
1	1001	t=a+b	有进位，cf=1	和为零，zf=1

			无进位，cf=0	和不为零，zf=0
1	0110	t=b-a	有借位，cf=1 无借位，cf=0	差为零，zf=1 差不为零，zf=0
1	1011	t=a&b	不影响	不影响
1	0101	t=/b(注：b 相反)	不影响	不影响
0	1010	t=b	不影响	不影响
0	1100 或 0100	t=a	不影响	不影响

三、实验过程

1、指令译码器

A）创建工程（选择的芯片为 family=Cyclone II；name=EP2C5T144C8）

步骤：左上角 file->New Project Wizard->选择工程位置和工程名->选择芯片 Cyclone II，available device 中选择 EP2C5T144C8->点击 next->最后点击 finish 完成创建工程

工程创建图：



B）编写源代码

根据实验指导和要求实现的功能写出对应的 Verilog 代码。

步骤：左上角 file->new->Verilog hdl file->编写代码（模块名需与工程名一致）->编译成功后保存到工程文件中

代码图：

```
module ins_decode(ir,en,mova,movb,movc,add,sub,andl,notl,rsr,rsl,jmp,jz,jc,inl,outl,nop,halt);
    input [7:0] ir;
    input en;
    output mova,movb,movc,add,sub,andl,notl,rsr,rsl,jmp,jz,jc,inl,outl,nop,halt;
    reg mova,movb,movc,add,sub,andl,notl,rsr,rsl,jmp,jz,jc,inl,outl,nop,halt;
    always@(ir,en)
    begin
    mova=1'b0;  movb=1'b0;  movc=1'b0;
    add=1'b0;   sub=1'b0;   andl=1'b0;
    notl=1'b0;  rsr=1'b0;   rsl=1'b0;
    jmp=1'b0;   jz=1'b0;    jc=1'b0;    inl=1'b0;
    outl=1'b0;  nop=1'b0;   halt=1'b0;
        if (en==1)
        begin
            if(ir[7:4]==4'bl100)
                begin
                    if(ir[3]&ir[2]) movb=1;
                    else if(ir[1]&ir[0]) movc=1;
                    else mova=1'bl;
                end
            else if(ir[7:4]==4'bl001) add=1;
            else if(ir[7:4]==4'b0110) sub=1;
            else if(ir[7:4]==4'bl011) andl=1;
            else if(ir[7:4]==4'b0101) notl=1;
            else if(ir[7:4]==4'bl010)
                begin
                    if(~ir[1] & ~ir[0]) rsr=1;
                    else rsl=1;
                end
            else if(ir[7:4]==4'b0011)
                begin
                    jc=ir[1];
                    jz=ir[0];
                    if (~ir[1] & ~ir[0]) jmp=1;
                end
            else if(ir[7:4]==4'b0010) inl=1;
            else if(ir[7:4]==4'b0100) outl=1;
            else if(ir[7:4]==4'b0111) nop=1;
            else if(ir[7:4]==4'bl000) halt=1;
            else ;
        end
        else ;
    end
endmodule
```

C) 编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

确定源代码文件为当前工程文件，点击【processing】-【start compilation】进行文件编译，编译成功，保存文件。

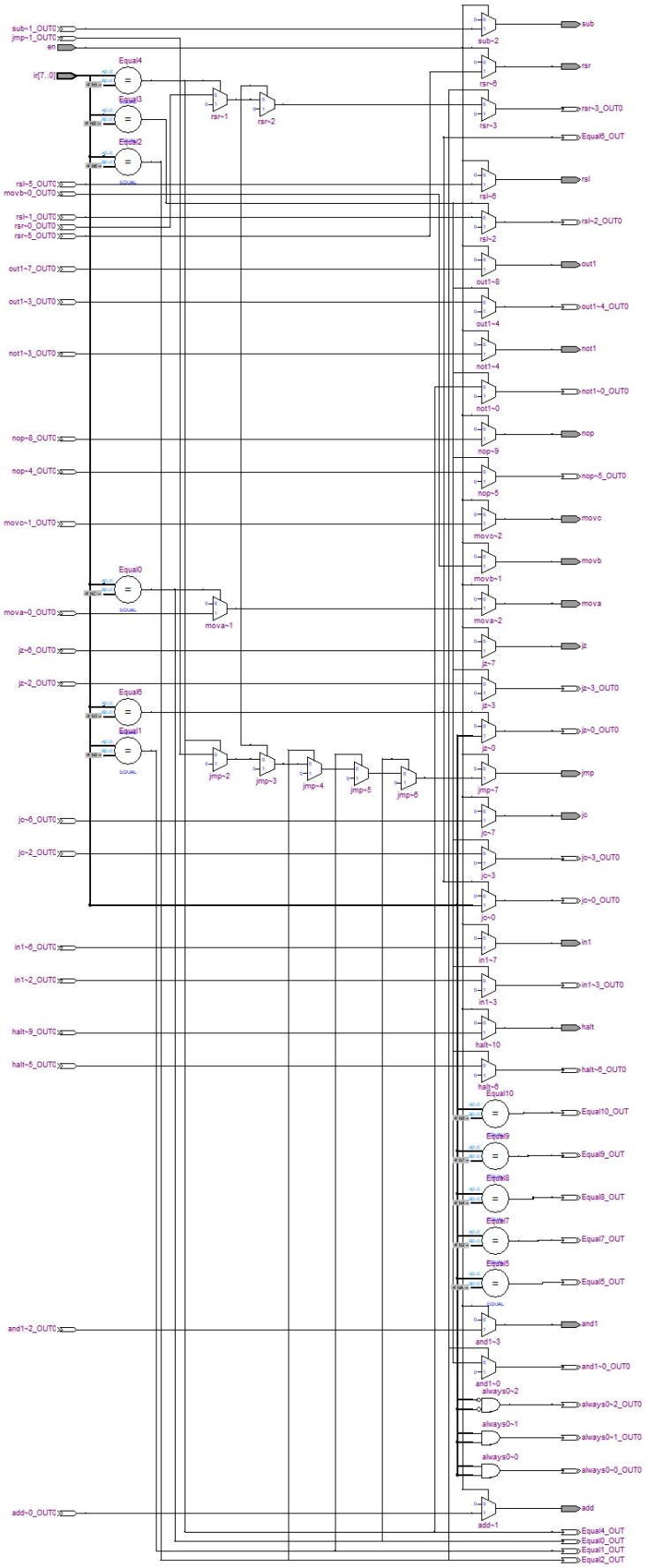
警告信息：

```
Warning: Feature LogicLock is not available with your current license
Warning: No exact pin location assignment(s) for 25 pins of 25 total pins
Warning: Found 16 output pins without output pin load capacitance assignment
Warning: The Reserve All Unused Pins setting has not been specified, and will default to 'As output driving ground'.
```

资源消耗：

Flow Status	Successful - Tue Nov 22 22:48:03 2022
Quartus II Version	9.0 Build 184 04/29/2009 SP 1 SJ Web Edition
Revision Name	myCPU
Top-level Entity Name	ins_decode
Family	Cyclone II
Device	EP2C5T144C8
Timing Models	Final
Met timing requirements	Yes
Total logic elements	29 / 4,608 (< 1 %)
Total combinational functions	29 / 4,608 (< 1 %)
Dedicated logic registers	0 / 4,608 (0 %)
Total registers	0
Total pins	25 / 89 (28 %)
Total virtual pins	0
Total memory bits	0 / 119,808 (0 %)
Embedded Multiplier 9-bit elements	0 / 26 (0 %)
Total PLLs	0 / 2 (0 %)

D) RTL 视图


sub~1_OUT0
jmp~1_OUT0
en
ir[7..0]
Equal4
Equal3
Equal2
rsl~5_OUT0
movb~0_OUT0
rsl~1_OUT0
rsr~0_OUT0
rsr~5_OUT0
out1~7_OUT0
out1~3_OUT0
not1~3_OUT0
nop~8_OUT0
nop~4_OUT0
movc~1_OUT0
Equal0
mova~0_OUT0
jz~6_OUT0
jz~2_OUT0
Equal6
Equal1
jc~6_OUT0
jc~2_OUT0
in1~6_OUT0
in1~2_OUT0
halt~9_OUT0
halt~5_OUT0
and1~2_OUT0
add~0_OUT0
rsr~1
rsr~2
mova~1
jmp~2
jmp~3
jmp~4
jmp~5
jmp~6
sub
sub~2
rsr
rsr~6
rsr~3_OUT0
rsr~3
Equal6_OUT
rsl
rsl~6
rsl~2_OUT0
rsl~2
out1
out1~8
out1~4_OUT0
out1~4
not1
not1~4
not1~0_OUT0
not1~0
nop
nop~9
nop~5_OUT0
nop~5
movc
movc~2
movb
movb~1
mova
mova~2
jz
jz~7
jz~3_OUT0
jz~3
jz~0_OUT0
jz~0
jmp
jmp~7
jc
jc~7
jc~3_OUT0
jc~3
jc~0_OUT0
jc~0
in1
in1~7
in1~3_OUT0
in1~3
halt
halt~10
halt~6_OUT0
halt~6
Equal10
Equal10_OUT
Equal9
Equal9_OUT
Equal8
Equal8_OUT
Equal7
Equal7_OUT
Equal5
Equal5_OUT
and1
and1~3
and1~0_OUT0
and1~0
always0~2
always0~2_OUT0
always0~1
always0~1_OUT0
always0~0
always0~0_OUT0
add
add~1
Equal4_OUT
Equal0_OUT
Equal1_OUT
Equal2_OUT

视图分析及结论:

分析：

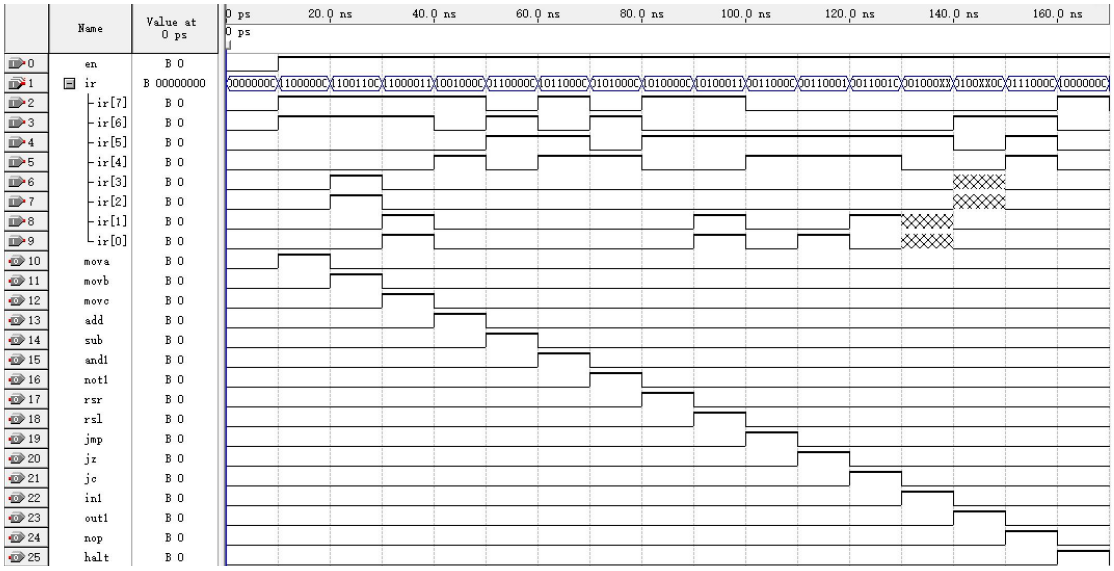
1、由视图可得，视图左边为输入，右边为输出。

2、其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；

3、还有许多 2-1 选择器，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 ir 和 en，输出信号包 括 add 等 16 种情况。各个输出端口之间通过导线相连。

结论:一个功能的实现需要经过多重门的处理后才能实现,一个元件的内部原理结构图十分复杂。

E) 功能仿真波形



结果分析及结论:

分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

当 en 为 0 时，不管 ir 为何值，16 个输出全为 0

当 en 为 1 时：

当 ir=11000000 时，mova 输出为 1；

当 ir=11001100 时，movb 输出为 1；

当 ir=11000011 时，movc 输出为 1；

当 ir=10010000 时，add 输出为 1；

当 ir=01100000 时，sub 输出为 1；

当 ir=10110000 时，and1 输出为 1；

当 ir=01010000 时，not1 输出为 1；

当 ir=10100000 时，rsr 输出为 1；

当 ir=10100011 时，rsl 输出为 1；

当 ir=00110000 时，jmp 输出为 1；

当 ir=00110001 时，jz 输出为 1；

当 ir=00110010 时，jc 输出为 1；

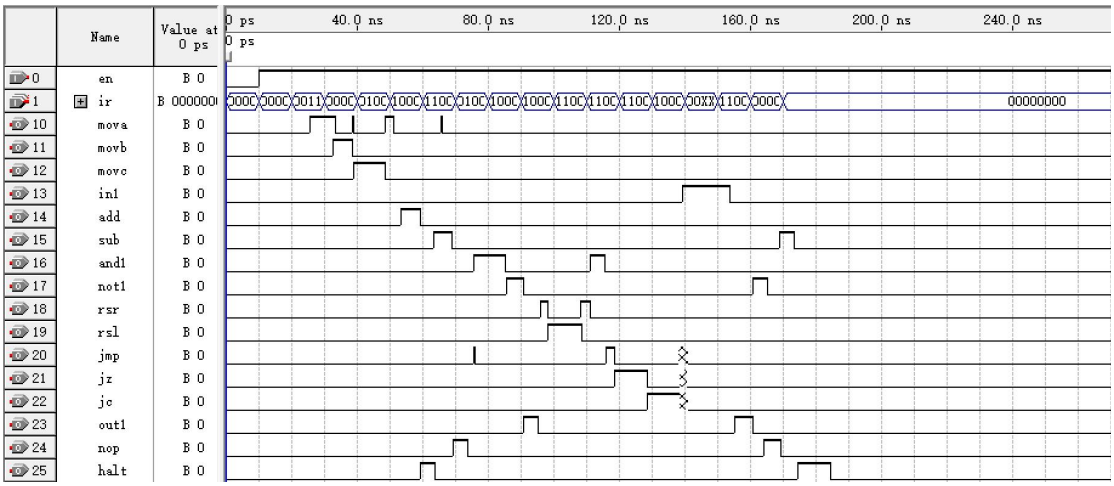
当 ir=00100000 时，in1 输出为 1；

当 ir=01000000 时，out1 输出为 1；

当 ir=01110000 时，nop 输出为 1；
当 ir=10000000 时，halt 输出为 1；

结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

F) 时序仿真波形



结果分析及结论:

分析：

1、时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。

2、由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。

结论：

1、时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。

2、时序仿真不仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

G) 时序分析

操作方法是：编译后，在 compilation report 中选择【timing analysis】-【summary】和【tpd】

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	16.272 ns	ir[7]	movc
2	N/A	None	16.055 ns	ir[5]	movc
3	N/A	None	16.052 ns	ir[6]	movc
4	N/A	None	15.903 ns	ir[7]	rsr
5	N/A	None	15.880 ns	ir[7]	jmp
6	N/A	None	15.848 ns	ir[7]	rsl
7	N/A	None	15.830 ns	ir[5]	jmp
8	N/A	None	15.792 ns	ir[5]	rsr
9	N/A	None	15.785 ns	ir[7]	mova
10	N/A	None	15.777 ns	ir[7]	jc
11	N/A	None	15.737 ns	ir[5]	rsl
12	N/A	None	15.727 ns	ir[5]	jc
13	N/A	None	15.649 ns	ir[6]	jmp
14	N/A	None	15.614 ns	ir[7]	and1
15	N/A	None	15.592 ns	ir[6]	rsr
16	N/A	None	15.568 ns	ir[5]	mova
17	N/A	None	15.565 ns	ir[6]	mova
18	N/A	None	15.564 ns	ir[7]	jz
19	N/A	None	15.546 ns	ir[6]	jc
20	N/A	None	15.537 ns	ir[6]	rsl
21	N/A	None	15.524 ns	ir[5]	and1
22	N/A	None	15.514 ns	ir[5]	jz
23	N/A	None	15.511 ns	ir[7]	not1
24	N/A	None	15.355 ns	ir[7]	out1
25	N/A	None	15.344 ns	ir[6]	not1
26	N/A	None	15.341 ns	ir[7]	movb
27	N/A	None	15.340 ns	ir[5]	not1
28	N/A	None	15.339 ns	ir[6]	and1
29	N/A	None	15.333 ns	ir[6]	jz
30	N/A	None	15.178 ns	ir[6]	out1
31	N/A	None	15.176 ns	ir[5]	out1
32	N/A	None	15.124 ns	ir[5]	movb
33	N/A	None	15.121 ns	ir[6]	movb
34	N/A	None	14.598 ns	ir[7]	halt
35	N/A	None	14.341 ns	en	halt
36	N/A	None	14.234 ns	en	movc
37	N/A	None	14.072 ns	ir[7]	nop
38	N/A	None	14.008 ns	ir[5]	nop
39	N/A	None	13.887 ns	ir[6]	nop
40	N/A	None	13.886 ns	ir[7]	in1
41	N/A	None	13.854 ns	ir[7]	add
42	N/A	None	13.849 ns	ir[2]	movc
43	N/A	None	13.820 ns	ir[5]	in1
44	N/A	None	13.813 ns	ir[5]	halt
45	N/A	None	13.726 ns	ir[6]	halt
46	N/A	None	13.637 ns	ir[5]	add
47	N/A	None	13.635 ns	ir[2]	mova
48	N/A	None	13.617 ns	ir[6]	in1
49	N/A	None	13.556 ns	ir[6]	add
50	N/A	None	13.541 ns	en	jc
51	N/A	None	13.527 ns	ir[7]	sub
52	N/A	None	13.460 ns	ir[5]	sub
53	N/A	None	13.388 ns	en	and1
54	N/A	None	13.355 ns	ir[6]	sub
55	N/A	None	13.153 ns	en	not1
56	N/A	None	13.153 ns	en	movb
57	N/A	None	13.150 ns	en	mova
58	N/A	None	13.110 ns	en	nop
59	N/A	None	13.110 ns	en	out1
60	N/A	None	13.068 ns	en	in1
61	N/A	None	13.034 ns	en	add
62	N/A	None	12.817 ns	en	jmp
63	N/A	None	12.803 ns	ir[2]	movb
64	N/A	None	12.781 ns	en	rsr
65	N/A	None	12.752 ns	en	jz
66	N/A	None	12.716 ns	en	rsl
67	N/A	None	12.711 ns	en	sub
68	N/A	None	11.701 ns	ir[4]	movc
69	N/A	None	11.324 ns	ir[4]	jmp
70	N/A	None	11.316 ns	ir[4]	rsr
71	N/A	None	11.261 ns	ir[4]	rsl
72	N/A	None	11.221 ns	ir[4]	jc
73	N/A	None	11.214 ns	ir[4]	mova
74	N/A	None	11.054 ns	ir[4]	and1
75	N/A	None	11.008 ns	ir[4]	jz
76	N/A	None	10.948 ns	ir[4]	not1
77	N/A	None	10.783 ns	ir[4]	out1
78	N/A	None	10.770 ns	ir[4]	movb
79	N/A	None	9.488 ns	ir[4]	nop
80	N/A	None	9.459 ns	ir[4]	halt
81	N/A	None	9.302 ns	ir[4]	in1
82	N/A	None	9.283 ns	ir[4]	add
81	N/A	None	9.302 ns	ir[4]	in1
82	N/A	None	9.283 ns	ir[4]	add
83	N/A	None	9.133 ns	ir[3]	movc
84	N/A	None	9.094 ns	ir[0]	mova
85	N/A	None	9.011 ns	ir[0]	movc
86	N/A	None	8.949 ns	ir[4]	sub
87	N/A	None	8.825 ns	ir[1]	mova
88	N/A	None	8.787 ns	ir[1]	movc
89	N/A	None	8.694 ns	ir[1]	jc
90	N/A	None	8.613 ns	ir[3]	mova
91	N/A	None	8.524 ns	ir[0]	jz
92	N/A	None	8.524 ns	ir[3]	movb
93	N/A	None	8.494 ns	ir[0]	jmp
94	N/A	None	8.478 ns	ir[0]	rsl
95	N/A	None	8.451 ns	ir[0]	rsr
96	N/A	None	8.358 ns	ir[1]	jmp

结果分析及结论:

分析：

1、由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个 定时特性的最坏情况定时。比如从 ir[3]到 jmp 的最坏定时情况的 tpd 为 11.720ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 ir[5]到 add 的 tpd 为 11.699ns。

结论：实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

2、算术逻辑单元 ALU

A）创建工程（选择的芯片为 family=FLEX10K；name=EPF10K20TI144-4）



B) 编写源代码

```
1  module simple(a,b,s,m,t,cf,zf);
2      input [7:0] a;
3      input [7:0] b;
4      input m;
5      input [3:0] s;
6      output cf,zf;
7      output [7:0] t;
8      reg cf,zf;
9      reg [7:0] t;
10     always@(m,s,a,b)
11     begin
12         t=8'b00000000;
13         cf=1'b0;
14         zf=1'b0;
15         if(m==1)
16         begin
17             if(s==4'b1001)
18             begin
19                 {cf,t}=a+b;
20                 if(t==0) zf=1'b1;
21                 else zf=1'b0;
22             end
23             else if(s==4'b0110)
24             begin
25                 {cf,t}=b-a;
26                 if(t==0) zf=1'b1;
27                 else zf=1'b0;
28             end
29             else if(s==4'b1011) t=a&b;
30             else if(s==4'b0101) t=~b;
31             else ;
32         end
33         else
34         begin
35             if(s==4'b1010) t=b;
36             else if (s==4'b1100 || s==4'b0100) t=a;
37             else ;
38         end
39     end
40 endmodule
```

C) 编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

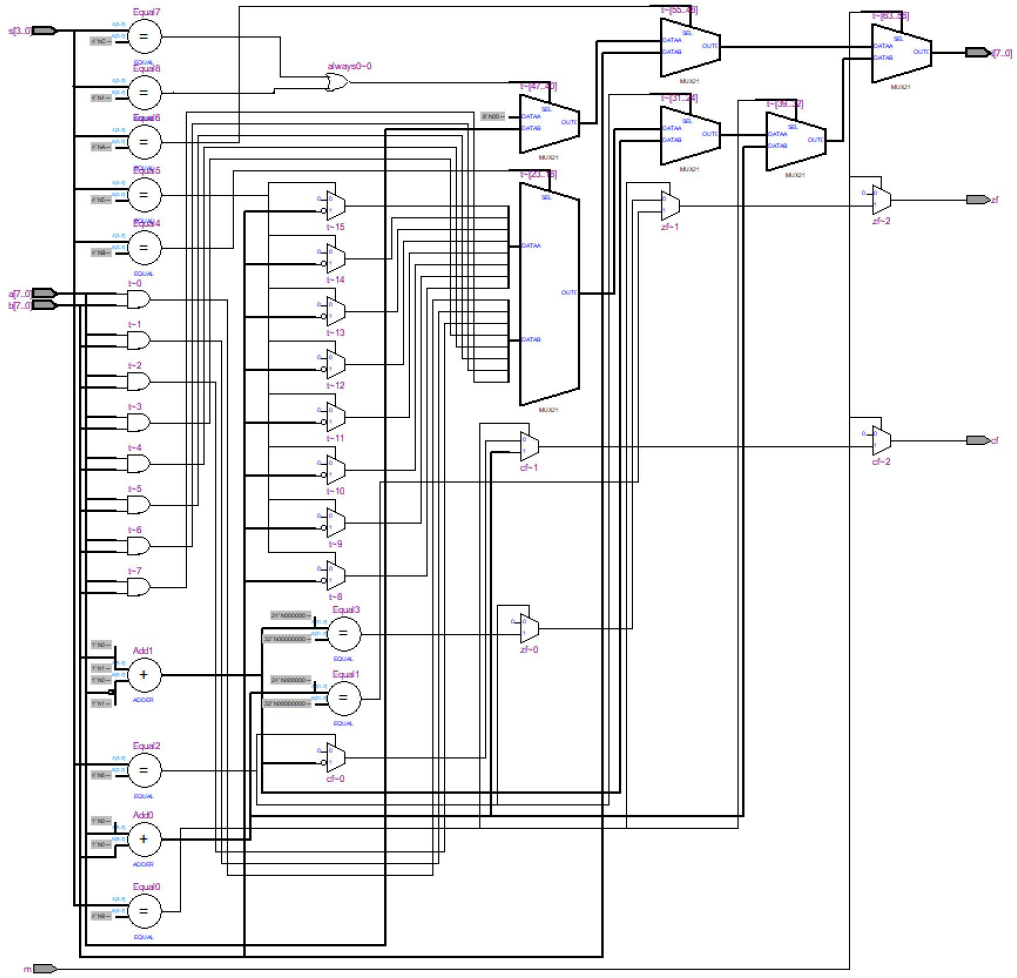
警告信息：



资源消耗：

Flow Status	Successful - Tue Nov 22 23:22:11 2022
Quartus II Version	9.0 Build 184 04/29/2009 SP 1 SJ Web Edition
Revision Name	simple
Top-level Entity Name	simple
Family	FLEX10K
Device	EPF10K20TI144-4
Timing Models	Final
Met timing requirements	Yes
Total logic elements	73 / 1,152 (6 %)
Total pins	31 / 102 (30 %)
Total memory bits	0 / 12,288 (0 %)

D) RTL 视图



结果分析及结论：

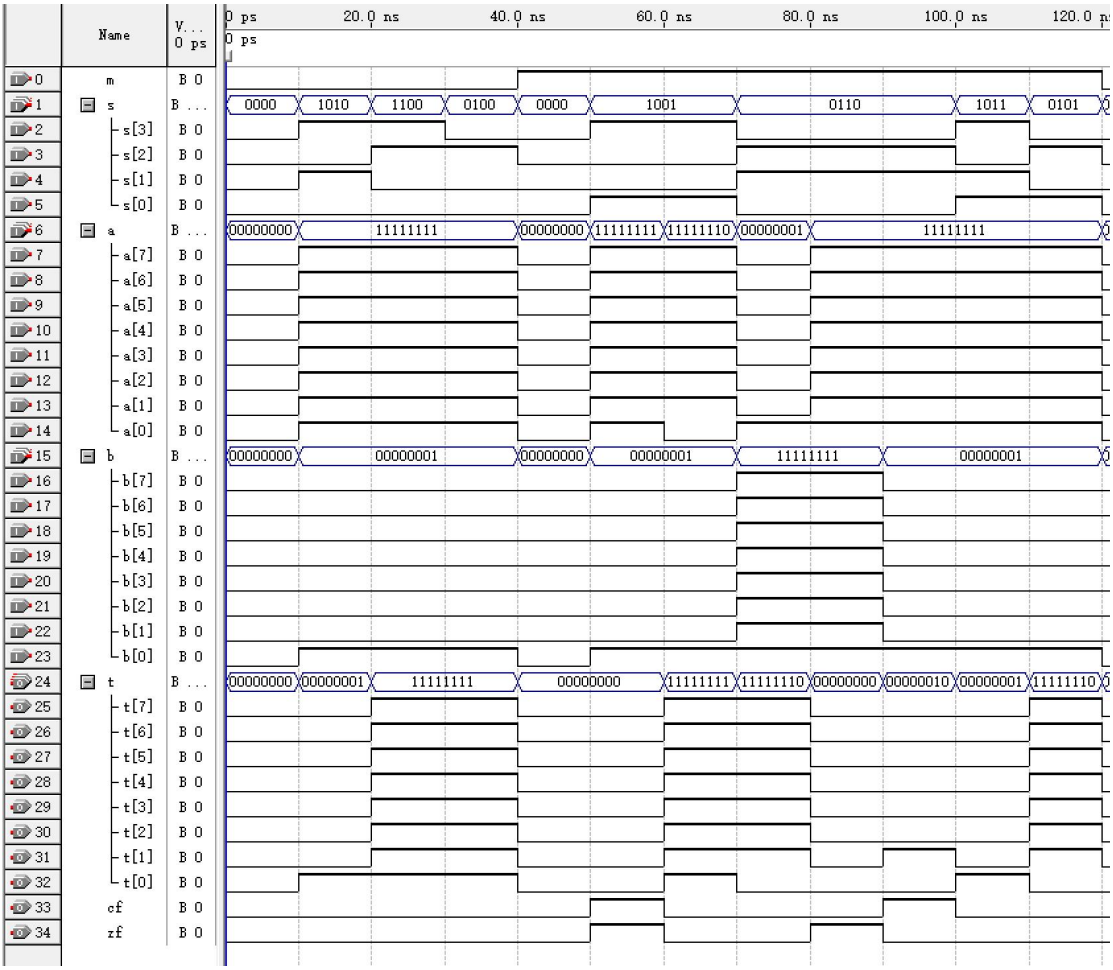
分析：

1、由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；

2、同时存在大量的 2-1 选择器，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 m，s，a，b，输出信号 为 t，cf，zf。各个输出端口之间通过导线相连。

结论：一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

E) 功能仿真波形



结果分析及结论:

分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

当控制信号 m 为 1，s 为 1001 时，执行 t=a+b

当控制信号 m 为 1，s 为 0110 时，执行 t=b-a

当控制信号 m 为 1，s 为 1011 时，执行 t=a&b

当控制信号 m 为 1，s 为 0101 时，执行 t=~b

当控制信号 m 为 1，s 为 1010 时，执行 t=b

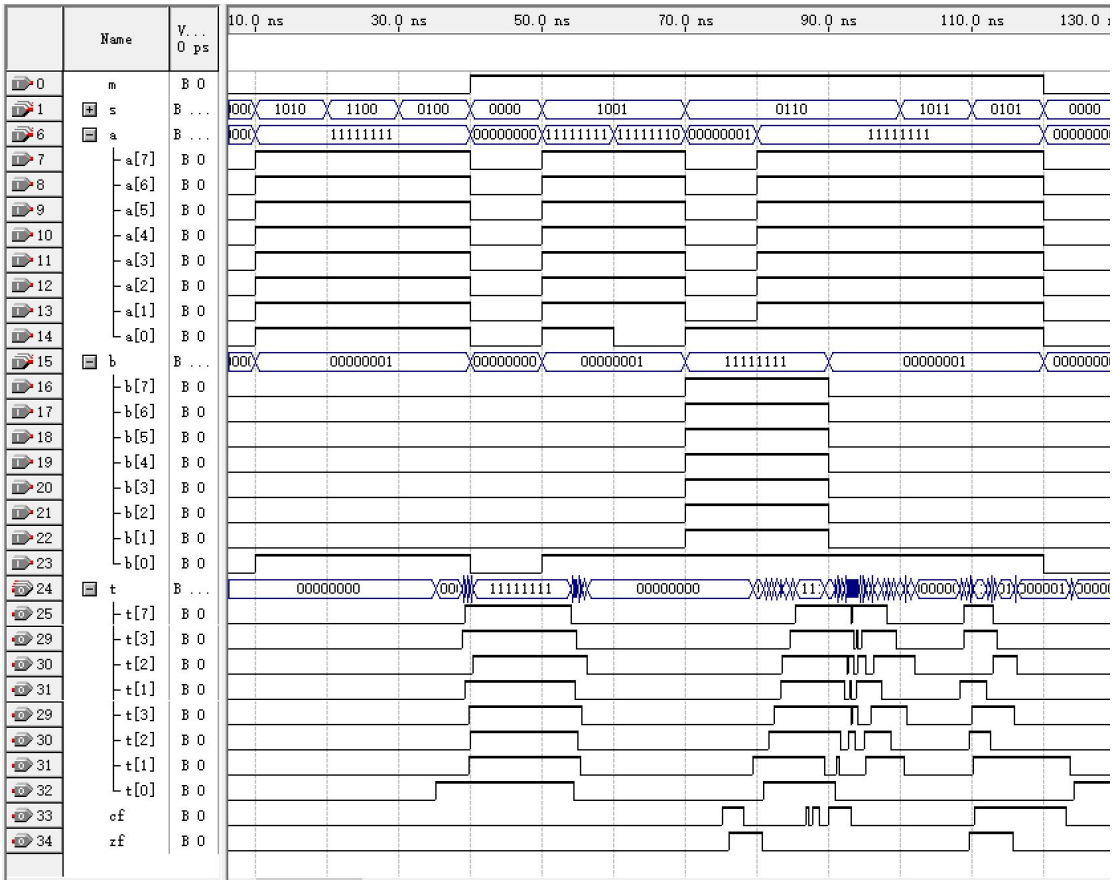
当控制信号 m 为 0，s 为 1100 时，执行 t=a

有进位和借位时 cf 为 1，否则为 0；

和为 0 或差为 0 时 zf 为 1，否则为 0；

结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

F）时序仿真波形



结果分析及结论:

分析：

1、时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。

2、由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。

结论：

1、时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。

2、时序仿真不 仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

G) 时序分析

Timing Analyzer Summary

	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths
1	Worst-case tpd	N/A	None	54.000 ns	b[0]	zf	--	--	0
2	Total number of failed paths								0

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	54.000 ns	b[0]	zf
2	N/A	None	53.500 ns	a[0]	zf
3	N/A	None	52.200 ns	a[1]	zf
4	N/A	None	51.900 ns	b[1]	zf
5	N/A	None	49.000 ns	b[2]	zf
6	N/A	None	48.000 ns	a[2]	zf
7	N/A	None	47.100 ns	b[0]	cf
8	N/A	None	46.600 ns	a[0]	cf
9	N/A	None	45.600 ns	b[0]	t[7]
10	N/A	None	45.300 ns	a[1]	cf
11	N/A	None	45.100 ns	a[0]	t[7]
12	N/A	None	45.000 ns	b[1]	cf
13	N/A	None	43.900 ns	a[3]	zf
14	N/A	None	43.800 ns	a[1]	t[7]
15	N/A	None	43.500 ns	b[1]	t[7]
16	N/A	None	43.100 ns	b[3]	zf
17	N/A	None	42.100 ns	b[2]	cf
18	N/A	None	41.600 ns	b[0]	t[6]
19	N/A	None	41.100 ns	a[2]	cf
20	N/A	None	41.100 ns	a[0]	t[6]
21	N/A	None	40.900 ns	b[0]	t[5]
22	N/A	None	40.600 ns	b[2]	t[7]
23	N/A	None	40.400 ns	a[0]	t[5]
24	N/A	None	39.800 ns	a[1]	t[6]
25	N/A	None	39.600 ns	a[2]	t[7]
26	N/A	None	39.500 ns	b[1]	t[6]
27	N/A	None	39.400 ns	b[4]	zf
28	N/A	None	39.100 ns	a[1]	t[5]
29	N/A	None	38.800 ns	b[1]	t[5]
30	N/A	None	38.400 ns	a[4]	zf
31	N/A	None	37.000 ns	a[3]	cf
32	N/A	None	36.600 ns	b[2]	t[6]
33	N/A	None	36.200 ns	b[3]	cf
34	N/A	None	36.000 ns	b[5]	zf
35	N/A	None	35.900 ns	b[2]	t[5]
36	N/A	None	35.800 ns	a[2]	t[6]
37	N/A	None	35.500 ns	a[3]	t[7]
38	N/A	None	35.100 ns	a[5]	zf
39	N/A	None	34.900 ns	a[2]	t[5]
40	N/A	None	34.700 ns	b[3]	t[7]
41	N/A	None	33.000 ns	b[0]	t[4]
42	N/A	None	32.500 ns	b[4]	cf
43	N/A	None	32.500 ns	a[0]	t[4]
44	N/A	None	31.800 ns	a[6]	zf
45	N/A	None	31.500 ns	a[4]	cf
46	N/A	None	31.500 ns	a[3]	t[6]
47	N/A	None	31.300 ns	b[6]	zf
48	N/A	None	31.200 ns	a[1]	t[4]
49	N/A	None	31.000 ns	b[4]	t[7]
50	N/A	None	31.000 ns	b[0]	t[3]
51	N/A	None	30.900 ns	b[1]	t[4]
52	N/A	None	30.800 ns	a[3]	t[5]
53	N/A	None	30.700 ns	b[3]	t[6]
54	N/A	None	30.600 ns	a[0]	t[3]
55	N/A	None	30.000 ns	a[4]	t[7]
56	N/A	None	30.000 ns	b[3]	t[5]
57	N/A	None	29.800 ns	s[3]	zf
58	N/A	None	29.200 ns	s[1]	zf
59	N/A	None	29.200 ns	a[1]	t[3]
60	N/A	None	29.100 ns	b[5]	cf
61	N/A	None	28.900 ns	b[1]	t[3]
62	N/A	None	28.500 ns	s[3]	t[5]
63	N/A	None	28.200 ns	a[5]	cf
64	N/A	None	28.000 ns	s[0]	zf
65	N/A	None	28.000 ns	b[2]	t[4]
66	N/A	None	27.800 ns	s[2]	zf
67	N/A	None	27.800 ns	s[3]	t[3]
68	N/A	None	27.600 ns	b[5]	t[7]
69	N/A	None	27.600 ns	s[3]	t[1]
70	N/A	None	27.500 ns	s[1]	t[5]
71	N/A	None	27.300 ns	b[7]	zf
72	N/A	None	27.300 ns	s[3]	t[0]
73	N/A	None	27.100 ns	s[3]	t[2]
74	N/A	None	27.000 ns	s[3]	t[6]
75	N/A	None	27.000 ns	b[4]	t[6]
76	N/A	None	27.000 ns	a[2]	t[4]
77	N/A	None	26.800 ns	s[1]	t[3]
78	N/A	None	26.700 ns	a[5]	t[7]
79	N/A	None	26.700 ns	s[0]	t[5]
80	N/A	None	26.700 ns	s[3]	t[4]
81	N/A	None	26.700 ns	s[1]	t[1]
82	N/A	None	26.600 ns	s[1]	t[2]
83	N/A	None	26.500 ns	a[7]	zf
84	N/A	None	26.500 ns	s[3]	t[7]
85	N/A	None	26.500 ns	s[2]	t[5]
86	N/A	None	26.300 ns	b[4]	t[5]
87	N/A	None	26.300 ns	s[1]	t[0]
88	N/A	None	26.000 ns	s[1]	t[6]
89	N/A	None	26.000 ns	a[4]	t[6]
90	N/A	None	26.000 ns	s[0]	t[3]
91	N/A	None	26.000 ns	b[2]	t[3]
92	N/A	None	25.900 ns	s[1]	t[7]
93	N/A	None	25.900 ns	s[1]	t[4]
94	N/A	None	25.800 ns	s[2]	t[3]
95	N/A	None	25.800 ns	s[0]	t[1]
96	N/A	None	25.600 ns	s[2]	t[1]
97	N/A	None	25.500 ns	s[0]	t[0]
98	N/A	None	25.300 ns	s[3]	cf
99	N/A	None	25.300 ns	a[4]	t[5]
100	N/A	None	25.300 ns	s[2]	t[0]
101	N/A	None	25.200 ns	s[0]	t[6]
102	N/A	None	24.900 ns	s[0]	t[4]
103	N/A	None	24.700 ns	s[1]	cf
104	N/A	None	24.700 ns	s[0]	t[7]
105	N/A	None	24.700 ns	s[2]	t[4]
106	N/A	None	24.700 ns	b[0]	t[2]
107	N/A	None	24.600 ns	s[2]	t[2]
108	N/A	None	24.500 ns	s[2]	t[7]
109	N/A	None	24.400 ns	b[0]	cf
110	N/A	None	24.200 ns	s[0]	t[2]
111	N/A	None	23.800 ns	s[0]	t[2]
112	N/A	None	23.600 ns	b[6]	t[6]
113	N/A	None	23.500 ns	m	zf
114	N/A	None	23.500 ns	s[0]	cf
115	N/A	None	23.500 ns	s[0]	t[7]
116	N/A	None	23.500 ns	s[2]	cf
117	N/A	None	23.300 ns	s[2]	cf
118	N/A	None	23.500 ns	a[6]	t[7]
119	N/A	None	23.300 ns	s[2]	cf
120	N/A	None	23.100 ns	b[6]	t[7]
121	N/A	None	23.100 ns	s[0]	t[0]
122	N/A	None	23.100 ns	a[3]	t[4]
123	N/A	None	23.000 ns	b[2]	t[2]
124	N/A	None	22.900 ns	b[7]	t[7]
125	N/A	None	22.900 ns	b[5]	t[5]
126	N/A	None	22.900 ns	s[1]	t[2]
127	N/A	None	22.700 ns	s[5]	t[6]
128	N/A	None	22.700 ns	b[6]	t[6]
129	N/A	None	22.600 ns	b[1]	t[2]
130	N/A	None	22.500 ns	m	t[5]
131	N/A	None	22.500 ns	s[3]	t[3]
132	N/A	None	22.300 ns	b[4]	t[4]
133	N/A	None	22.300 ns	b[0]	t[1]
134	N/A	None	22.200 ns	a[7]	t[7]
135	N/A	None	22.100 ns	a[5]	t[5]
136	N/A	None	22.100 ns	b[3]	t[4]
137	N/A	None	22.000 ns	s[2]	t[2]
138	N/A	None	21.800 ns	m	t[3]
139	N/A	None	21.800 ns	a[0]	t[1]
140	N/A	None	21.600 ns	m	t[1]
141	N/A	None	21.500 ns	b[3]	t[3]
142	N/A	None	21.300 ns	s[4]	t[4]
143	N/A	None	21.300 ns	m	t[2]
144	N/A	None	21.300 ns	m	t[0]
145	N/A	None	21.000 ns	m	t[6]
146	N/A	None	20.700 ns	m	t[4]
147	N/A	None	20.600 ns	m	t[7]
148	N/A	None	20.600 ns	a[1]	t[1]
149	N/A	None	20.400 ns	b[7]	cf
150	N/A	None	20.200 ns	b[1]	t[1]
151	N/A	None	20.000 ns	b[0]	t[0]
152	N/A	None	20.000 ns	a[0]	t[0]
153	N/A	None	19.600 ns	a[7]	cf
154	N/A	None	19.200 ns	m	cf

结果分析及结论:

分析：

1、由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个 定时特性的最坏情况定时。比如从 a[0]到 zf 的最坏定时情况的 tpd 为 38.50ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 s[0]到 t[0]的 tpd 为 29.6ns。

结论：实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

四、思考题

1．指令译码器必须要 16 个输出吗？可否将一些输出合并，哪些可以合并，为什么？

答：不一定。jmp 和 add 可以合并起来，因为 jmp 是将 add 后的结果写入 pc 中，则可以进行 add 操作后直接进行写入操作。Add 和 sub 和 and 操作可以合并，因为这三个操作类似，且输出为使能信号，故可以用一个合并使能信号来作为三个输出的共同使能信号。

2．ALU 中的 S[3..0]控制信号是来自哪里或者说与什么信息相同？

答：来自指令码 ir 的前四位。

3、为何 S[3..0]等于 1100 或 0100 时将输入 a 传给 t，S[3..0]等于 1010 时将输入 b 传给 t？

答：S[3..0]为控制信号，当 S[3..0]输入为 1100 时，控制输出 t 等于 a，S[3..0]等于 1010 或 0100 时，t 等于 b，此时 alu 相当于选择器。

五、实验总结、必得体会及建议

1、从需要掌握的理论、遇到的困难、解决的办法以及经验教训等方面进行总结。

（1）需要掌握的理论：基本了解了简易模型机的内部结构和工作原理。同时熟悉了译码器、ALU 的工作原理。学会 使用 Verilog 语言编写电路。

（2）遇到的困难：对于 QuartusII 的使用还不够熟练，特别是进行波形仿真的功能仿真和时许仿真分别怎么操作的方面有一定不足。

（3）解决方法：通过上网查询相关资料和询问同学后得以解决问题，并通过分析报告发现电路中的问题。有不理解的还请教了老师，不仅收获了方法还掌握的技巧。

（4）经验教训：对于电子电路的学习一定要肯动手，光是看是学不会的，一定要落到实处，多自己使用软件进行仿真，才能加深对于这门课程的理解。

2、对本实验内容、过程和方法的改进建议（可选项）。

1、对于“无影响”的变量，是否可以规定它的输出以实现更好的统一以及传递一些信息。

2、这两个模块是否还可以添加更多的功能以更贴合市场的需求。